C++ PROGRAMMING TEXTBOOK NOTES

Module: Method Overriding & Runtime Polymorphism

Author: Gaurav Kandpal | **Platform:** CS-Simplified | **Level:** Beginner to Advanced

1. CONCEPTUAL FOUNDATIONS OF METHOD OVERRIDING

What is Method Overriding?
Method Overriding occurs when a **derived (child) class** redefines a member function that has the **exact same name, return type, and signature (parameter list)** as a function declared in its **base (parent) class**. When invoked through a child object, the child's version hides/overrides the parent's implementation.

Essential Conditions for Method Overriding:

- **Inheritance Requirement:** Must involve an active base-to-derived class relationship (Is-A relationship).
- **Identical Signature:** Function name, parameter types, number of parameters, and order must be 100% identical.
- **Identical Return Type:** Return types must match (or be covariant pointer/reference types).
- **Scope Difference:** Parent version resides in Parent class scope; Child version resides in Child class scope.

```cpp
// C++ Method Overriding Demonstration (Static Binding)
#include <iostream>
using namespace std;

class Parent {
public:
    void show() {
        cout << "This is Parent class show() ";
    }
};

class Child : public Parent {
public:
    // Function Overriding: Same name & parameters as Base
    void show() {
        cout << "This is Child class show() ";
    }
};

int main() {
    Child obj;
    obj.show(); // Calls Child's show() via direct object

    Parent parentObj;
    parentObj.show(); // Calls Parent's show()

    // Pointer Execution Test
    Parent* ptr = &obj;
    ptr->show(); // CALLS PARENT SHOW() due to Static Binding!

    return 0;
}
```

2. OBJECT VS. POINTER BINDING MECHANICS

Direct Object Access vs. Base Pointer Access:

When objects are accessed **directly by value** (e.g., Child obj; obj.show();), C++ uses **Early Binding (Static Binding)** at compile time based strictly on the declared object type. If you instantiate a Parent p; p.show();, it executes Parent::show(). Instantiating concrete objects for every derived variation lacks flexibility and prevents writing scalable polymorphic code.

REAL-WORLD ANALOGY: Universal Remote Control & Electronics

- **Base Class Pointer (The Remote):** Think of a base class pointer as a Universal Remote Control.
- **Child Objects (Devices):** The underlying hardware devices connected to it are TV, AC, or Fan.
- **Virtual Function (The Power Button):** You press *ONE single universal button* ('Power'). You don't care which physical device is wired.
- **Dynamic Polymorphism:** Pressing 'Power' executes TV Power on a TV object, AC Power on an AC object, and Fan speed on a Fan object.
- *Without base pointers and dynamic binding, you would need a completely separate remote for every single appliance!*

3. RUNTIME POLYMORPHISM & THE VIRTUAL KEYWORD

To implement true **Runtime Polymorphism (Late / Dynamic Binding)** in C++, two critical components are mandatory:
1. Declaring member functions as **virtual** in the Base Class.
2. Accessing derived objects through a **Base Class Pointer (Base*)** or **Base Class Reference (Base&)**.

```
// Dynamic Dispatch Implementation using virtual Keyword
#include <iostream>
using namespace std;

class Parent {
public:
    virtual void show() { // Virtual enables late binding
        cout << "Parent show() ";
    }
    virtual ~Parent() {} // Virtual destructor
};

class Child : public Parent {
public:
    void show() override { // Explicit C++11 override specifier
        cout << "Child show() ";
    }
};

int main() {
    Parent* ptr = new Child(); // Base pointer holding Derived object
    ptr->show(); // Dynamic Dispatch -> Calls Child::show()!

    delete ptr;
    return 0;
}
```

4. UNDER THE HOOD: VPTR & VTABLE ARCHITECTURE

When a class contains or inherits virtual functions, the compiler creates special dynamic lookup structures:
- **VTABLE (Virtual Table):** A static lookup array of function pointers created per class. The Derived class VTABLE replaces the Parent function address with the overridden Derived function address.
- **VPTR (Virtual Pointer):** A hidden pointer added to every object instance. During object instantiation in RAM, vptr is initialized to point directly to the class's VTABLE.

RAM STACK / HEAP (Caller Object Instance)	VTABLE IN READ-ONLY CODE SEGMENT

Child Object Memory Address: 0x7FFD10 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ **vptr** ■■■■ points to 0x800400 ■ ■ Child Member Variables... ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■	**Child Class VTABLE Location:** 0x800400 ■■■■■■■■■■■■■■■■■■■